

Relatório de Evolução do App ObservaATT PE

Atualizado em 22/04/2026.

1. Resumo Executivo

O ObservaATT PE evoluiu de uma aplicação web local para uma solução operacional com backend Flask, persistência híbrida, integração com Supabase, deploy em produção no Render, instalação como PWA e preparação de empacotamento iOS via Capacitor.

No estado atual, o sistema já suporta registro de sinistros com geolocalização, fotos, sincronização com banco remoto, exportações automáticas, visualização simplificada de registros e atualização forçada de clientes instalados por versionamento de Service Worker.

2. Situação Atual

- Nome do app: ObservaATT PE
- Backend: Flask em produção
- Banco principal: Supabase, tabela acidentes
- Cliente instalável: PWA ativo
- Versionamento atual do app: 1.0.3
- Versionamento atual do PWA/Service Worker: v13
- Empacotamento iOS: configurado via Capacitor
- Ambiente de deploy: Render com automação por GitHub Actions

3. Linha de Evolução

Fase 1. Base funcional do produto

- Construção da interface web principal para cadastro de sinistros.
- Suporte a geolocalização automática e envio de fotos.
- Criação de registros em JSON local como espelho operacional.
- Estrutura inicial PWA para instalação em dispositivos móveis.

Fase 2. Operação online e persistência

- Estruturação do backend Flask para servir frontend e API.
- Organização do fluxo de leitura e gravação dos acidentes.
- Implantação do modo híbrido com cache local e sincronização remota.
- Integração com Supabase como armazenamento principal.

Fase 3. Confiabilidade de produção

- Ajustes no startup do servidor para evitar travamento por operações remotas no boot.
- Isolamento das tarefas de inicialização em thread de fundo.
- Aumento de tolerância do Gunicorn para subida estável em produção.
- Inclusão de diagnósticos de saúde e status administrativo do Supabase.

Fase 4. Automação de deploy

- Criação e refinamento do workflow de deploy no GitHub Actions.
- Suporte a deploy hook do Render com sanitização e fallback automático.
- Estratégia de disparo resiliente por hook, service id e nome do serviço.
- Publicação contínua estabilizada no repositório principal.

Fase 5. Dados, exportações e recuperação

- Criação de exportações CSV diárias.
- Criação de mapa diário de Pernambuco com pontos dos acidentes.
- Processo de sincronização local para Supabase com upsert por id.
- Recuperação e migração de registros antigos do app local para a tabela acidentes.

Fase 6. Evolução da experiência do usuário

- Simplificação da visualização de registros.
- Remoção de colunas excedentes da tabela pública.
- Exibição limitada a quatro campos operacionais:
 - Data/Hora
 - Município
 - Veículo/Usuário
 - Tempo de registro
- Remoção da exibição de fotos na grade de registros.

Fase 7. Atualização forçada dos clientes

- Adoção de versionamento explícito do Service Worker e do manifest.
- Forçamento de refresh dos clientes instalados por invalidação de cache.
- Último ciclo aplicado com PWA v13 para propagação da versão atual a todos os usuários.

Fase 8. Preparação mobile iOS

- Configuração de Capacitor com appId br.gov.pe.observepe.
- Estrutura preparada para abrir a aplicação iOS apontando para a URL HTTPS de produção.
- Documentação específica de publicação para App Store já incluída no projeto.

4. Entregas Consolidadas

- bullet Cadastro de sinistros com localização e fotos.
- bullet Backend Flask operacional.
- bullet Integração com Supabase.
- bullet Persistência local como fallback.
- bullet Exportação CSV diária.
- bullet Mapa diário HTML.
- bullet Diagnóstico administrativo de saúde do Supabase.
- bullet Deploy automatizado no Render.
- bullet PWA instalável com atualização forçada.
- bullet Visualização pública simplificada de registros.
- bullet Preparação de empacotamento iOS com Capacitor.

5. Marcos Recentes Confirmados

- bullet Ajuste da tabela de registros para quatro colunas sem fotos.
- bullet Migração de registros antigos do app local para o banco acidentes.
- bullet Geração de relatório CSV de auditoria dos registros migrados.
- bullet Publicação da versão 1.0.3.
- bullet Ativação do cache versionado observatt-pe-v13.

6. Benefícios Obtidos

- bullet Menor risco de perda de dados em produção.
- bullet Melhor estabilidade de deploy e reinicialização.
- bullet Melhor legibilidade operacional da tela de registros.
- bullet Capacidade de atualização coordenada de todos os clientes instalados.
- bullet Base pronta para expansão mobile nativa no ecossistema iOS.

7. Próximos Passos Recomendados

- bullet Consolidar um painel administrativo com filtros por período e município.
- bullet Adicionar autenticação mais forte para rotas administrativas.
- bullet Automatizar geração de relatórios executivos periódicos.
- bullet Refinir o empacotamento iOS até publicação final na App Store.
- bullet Ampliar rastreabilidade operacional com logs funcionais e auditoria de sincronização.

8. Conclusão

O projeto já se encontra em estágio operacional sólido. A solução deixou de ser apenas um formulário web simples e passou a operar como plataforma de registro, consulta, exportação e sincronização de

sinistros, com infraestrutura de produção, atualização forçada dos clientes e caminho aberto para distribuição mobile.